

Configuración del proxy frente al dashboard

Qué es `trust proxy`, para qué se usa exactamente, y por qué la configuración de nginx en producción puede dejar sin login a toda la empresa o debilitar el control de intentos de acceso si no está bien hecha.

1 El escenario real

El sistema no se expone directo a internet. En producción hay un proxy (nginx, o equivalente — Caddy/Cloudflare cumplirían el mismo rol) delante del servidor Node de la aplicación:

```
Navegador del usuario → HTTPS → nginx (proxy) → HTTP interno → node server.js
```

El navegador habla HTTPS con nginx, pero nginx habla con la aplicación por dentro de la red (HTTP plano, sin cifrar, porque ya está dentro del mismo servidor o red privada). Esto es estándar y correcto — el problema es que **Express (el framework del servidor) no sabe, por sí solo, que existe ese salto**: solo ve la conexión que le llega desde nginx, que es HTTP local, y por defecto asume que todo el tráfico llegó sin cifrar.

2 Qué hace `trust proxy`

En el archivo principal del servidor:

```
app.set('trust proxy', 1); // server.js, línea 42
```

Esta línea le dice a Express: *"confía en 1 salto de proxy delante de ti, y lee las cabeceras `X-Forwarded-*` que ese proxy agrega para saber la verdad de la petición original"*. Dos cabeceras concretas entran en juego:

- **X-Forwarded-Proto** — le dice a Express si la conexión original (navegador → nginx) fue `https` o `http`. Sin esto, Express siempre cree que fue `http`.
- **X-Forwarded-For** — le dice cuál es la IP real del navegador que hizo la petición. Sin esto, Express ve como IP de origen la IP interna de nginx — la misma para todo el mundo, porque todos pasan por el mismo proxy.

3 Para qué se usa cada cabecera, exactamente

3.1 • **X-Forwarded-Proto** → la cookie de sesión (login)

server.js, líneas 106–119

```
app.use(session({
  name: 'gtc.sid',
  secret: SESSION_SECRET,
  cookie: {
    httpOnly: true,
    sameSite: 'lax',
    secure: String(process.env.COOKIE_SECURE).toLowerCase() === 'true',
    maxAge: 1000 * 60 * 60 * 8,
  },
}));
```

Cuando `COOKIE_SECURE=true` (que es como debe estar en producción, detrás de HTTPS), la cookie de sesión (`gtc.sid`) se marca como `Secure`. Esa marca es una instrucción para el navegador: "nunca me mandes esta cookie si la conexión no es HTTPS". Es una protección real — evita que la cookie de sesión viaje en texto plano si alguien intentara acceder por HTTP.

El problema es cómo decide `express-session` si la conexión "fue HTTPS": lo hace mirando lo que Express cree que fue el protocolo original, y eso depende exclusivamente de la cabecera `X-Forwarded-Proto` — que Express solo lee si `trust proxy` está activado.

QUÉ PASA SI ESTO FALLA

Si `trust proxy` no estuviera activo, o si nginx no mandara bien esa cabecera: Express creería que *toda* conexión es HTTP plano → la cookie `Secure` nunca se envía porque el navegador la descarta → **nadie puede iniciar sesión en producción**, así el usuario y la contraseña sean correctos.

3.2 • X-Forwarded-For → el limitador de intentos de login

`src/middleware/rate-limit.js`, líneas 36–38

```
function clavesDe(req) {
  const ip = req.ip || req.socket?.remoteAddress || 'desconocida';
  const identificador = String(req.body?.email || '').trim().toLowerCase();
  const claves = [`ip:${ip}`];
  if (identificador) claves.push(`user:${identificador}`);
  return claves;
}
```

`req.ip`, en Express, con `trust proxy` activado, lee la IP real del navegador desde `X-Forwarded-For` (en vez de leer la IP de nginx, que sería la misma para todos los usuarios). El limitador de login bloquea 15 minutos cualquier combinación de **IP + usuario** que falle 8 veces seguidas, para frenar a alguien que intenta adivinar contraseñas por fuerza bruta.

4 Dónde está el riesgo real

El código del dashboard ya hace lo correcto: `trust proxy` está activado, y `req.ip` se usa de forma consistente en todo el sistema. Lo que Express **no puede verificar por sí solo** es si la cabecera `X-Forwarded-For` que le llega es confiable — Express confía ciegamente en que quien está delante (nginx) la puso bien, y no la dejó pasar tal cual venía del navegador del cliente. Esto quedó documentado como el hallazgo **QA-03** en la revisión de pruebas del sistema.

Hay dos formas de configurar nginx, y la diferencia es crítica:

CONFIGURACIÓN DE NGINX	QUÉ PASA EN LA PRÁCTICA
Correcta nginx sobrescribe <code>X-Forwarded-For</code> con la IP real de quien se conectó	El limitador funciona como se diseñó: cuenta los intentos fallidos por persona real. Un atacante o un empleado distraído solo se bloquea a sí mismo.
Incorrecta — no configurada nginx no agrega ni sobrescribe la cabecera	Express ve la IP interna de nginx para <i>todo el mundo</i> . 8 intentos fallidos de cualquier empleado , en cualquier parte de la empresa, bloquean el login de toda la empresa durante 15 minutos.
Incorrecta — sin filtrar nginx deja pasar la cabecera tal como la manda el cliente	Un atacante podría enviar un <code>X-Forwarded-For</code> distinto en cada intento (falsificado) para simular que cada intento fallido viene de una persona distinta, y saltarse por completo el límite de 8 intentos.

5 Por qué esto no se arregla en el código

El límite de 8 intentos, la ventana de 15 minutos, y la lectura de `req.ip` ya están implementados y probados (`test/integration/rate-limit-login.test.js` lo confirma contra un servidor real). Esa prueba demuestra que el limitador funciona correctamente *dado que* la cabecera `X-Forwarded-For` le llega bien armada — pero ninguna prueba automática de este repositorio puede saber si el nginx real de producción de GTC la está mandando bien, porque esa configuración **vive en el servidor de producción, fuera del código de la aplicación**.

6 Qué hay que verificar, paso a paso

1. Abrir la configuración de nginx (o el proxy que esté corriendo) en el servidor de producción de GTC.
2. Ubicar el bloque `location` que apunta al dashboard de Costos, y confirmar que fije estas dos cabeceras con el valor real (no que las deje pasar sin control):

```
proxy_set_header X-Forwarded-For $remote_addr;  
proxy_set_header X-Forwarded-Proto $scheme;
```

(la sintaxis exacta puede variar según cómo esté armado el server block, pero la idea siempre es la misma: nginx debe **fijar** el valor real, no reenviar ciegamente lo que traiga el cliente).

3. Si en algún punto de la cadena hay **más de un proxy** (por ejemplo, Cloudflare delante de nginx), el número `1` en `app.set('trust proxy', 1)` tendría que subir al número real de saltos — de lo contrario Express seguiría leyendo el salto equivocado de la cabecera.
4. Hacer una prueba real: fallar el login 8 veces desde una máquina y confirmar que solo esa persona queda bloqueada, no el resto de la empresa.

Este anexo acompaña el informe "Datos pendientes de Costeo" — punto 7, "Configuración del proxy (nginx)". GTC · Sistema de Costos.